

PRÁCTICA 2

Integrantes:

Alvarado Canchihuaman Hans Piero

Solis Vilcapoma Jeremy Miguel

Rojas Lujan Fabricio Sideral

1. Situación problemática

Un sistema registra distintos vehículos. El programador considera que Auto y Bicicleta son vehículos, por lo que ambas clases heredan de Vehiculo. Sin embargo, la clase Vehiculo obliga a todos sus hijos a encender un motor.

Analice el siguiente código. No lo modifique todavía.

```
class Vehiculo {
    public void encenderMotor() {
        System.out.println("Motor encendido");
    }
}
class Auto extends Vehiculo {
    @Override
    public void encenderMotor() {
        System.out.println("Motor del auto encendido");
    }
}
class Bicicleta extends Vehiculo {
    @Override
    public void encenderMotor() {
        throw new UnsupportedOperationException(
            "La bicicleta no tiene motor"
        );
    }
}
```

2. Código de prueba

```
public class Main {
    public static void probarVehiculo(Vehiculo vehiculo) {
        vehiculo.encenderMotor();
    }
    public static void main(String[] args) {
        probarVehiculo(new Auto());        probarVehiculo(new
        Bicicleta());
    }
}
```

3. Actividades que debe realizar

Actividad 1. Predicción

Sin ejecutar el programa, escriba qué cree que ocurrirá en cada llamada.

a) Resultado esperado con `new Auto()`:

Se ejecutará correctamente y mostrará:

Motor del auto encendido

b) Resultado esperado con `new Bicicleta()`:

Se producirá una excepción `UnsupportedOperationException` porque la bicicleta no tiene motor. La bicicleta no tiene motor El programa se detendrá en esa llamada debido a la excepción.

Actividad 2. Ejecución

Copie las clases en su entorno de desarrollo, compile y ejecute el programa. Después, registre el resultado real.

Salida obtenida con `Auto`:

Motor del auto encendido

Salida o error obtenido con `Bicicleta`:

Exception in thread "main" java.lang.UnsupportedOperationException: La bicicleta no tiene motor El Auto funciona correctamente, pero la Bicicleta genera un error porque no tiene motor.

Actividad 3. Identificación del problema

Responda con sus propias palabras:

¿Cuál es la clase padre?

la clase padre es vehículo

¿Cuáles son las clases hijas?

Las clases hijas son **Auto** y **Bicicleta**.

¿Qué clase ocasiona el error?

La clase que ocasiona el error es **Bicicleta**.

¿Por qué Bicicleta no puede cumplir correctamente el método `encenderMotor()`?

Porque una bicicleta **no tiene motor**, por lo que no puede realizar correctamente ese método y lanza una excepción.

¿Bicicleta puede reemplazar a Vehículo sin que el programa falle?

No, porque al utilizar `Bicicleta` como `Vehiculo`, el programa falla cuando se llama a `encenderMotor()`.

¿El código cumple o viola el Principio de Sustitución de Liskov? Explique.

Viola el Principio de Sustitución de Liskov (LSP), porque `Bicicleta` no puede sustituir correctamente a `Vehiculo`. Al llamar a `encenderMotor()`, la bicicleta produce un error en lugar de cumplir con el comportamiento esperado de la clase padre.